

CSCE 669 COMPUTATIONAL OPTIMIZATION · MODULE 13 · LECTURE NOTES

Solvers and Shipping

Using the tools, and saying what you solved.

MODULE OUTCOMES

- Choose a solver and a modelling layer.
- Diagnose numerical trouble.
- Explain what has actually made solvers fast.
- Decide between exact and heuristic approaches.
- State what an optimisation result can honestly promise.

1 The stack

Layer	Options	Note
Modelling layer	CVXPY, JuMP (Julia), Pyomo, AMPL, GAMS.	You write the problem; the layer writes the solver input. See the callout.
Linear and mixed-integer	HiGHS (free and genuinely good), Gurobi, CPLEX, SCIP.	The commercial solvers are 10 to 100× faster on hard instances — a real gap, from decades of cut generation, presolve, and heuristics (Module 09 §4). Academic licences are free.
Conic and semidefinite	SCS, ECOS, Clarabel, Mosek.	The free options are adequate at moderate sizes; Mosek is substantially better on large semidefinite programs (Module 10 §3).
Nonlinear	IPOPT (free), KNITRO, SNOPT.	Interior point methods (Module 07 §3), with the caveat that non-convex problems get local answers.
Constraint programming	OR-Tools CP-SAT.	A different paradigm — propagation and clause learning rather than relaxation — and it beats mixed-integer solvers on many scheduling and sequencing problems. Worth trying on any problem that is mostly combinatorial.
Local search	Simulated annealing, tabu search, large neighbourhood search.	No bound, so no guarantee — use when nothing else fits, and still compute a bound separately (§3).

USE A MODELLING LAYER

Write the problem in CVXPY, JuMP, or Pyomo and let it construct the solver's input, rather than assembling constraint matrices by hand — which is error-prone, unreadable, and ties you to one solver.

You can then change solvers in a single line, which makes comparison trivial and prevents lock-in to a commercial licence you may later lose. **Trying three solvers on a hard instance costs minutes and is frequently decisive.**

And the modelling layer catches errors. CVXPY's DCP rules (Module 02 §2) **reject a non-convex formulation at build time**, rather than passing it to a solver that will silently return a local answer presented as optimal — which is exactly the failure mode that makes non-convex results untrustworthy.

The cost is a translation overhead on very large models, which becomes significant only at a scale where you would be constructing the matrices deliberately and carefully anyway. **For everything else the layer is free.**

2 Numerical trouble

- **Different solvers disagree on the objective value.** The clearest possible signal that something is numerically wrong, and the reason to keep more than one solver available.
- **'Optimal' solutions that violate constraints by small amounts.** Solvers work to a feasibility tolerance, so a violation of 10^{-6} is expected — but if your constraint represents something that must hold exactly, the tolerance is a correctness problem and must be handled explicitly.
- **Infeasibility reported on a problem you know to be feasible**, or feasibility reported on one you know is not.
- **Results that change when you reorder the constraints**, which should be irrelevant and is a reliable sign of ill-conditioning.
- **And a huge range of coefficient magnitudes — the usual cause, and the first thing to check.** Solvers report a condition estimate and a coefficient range statistic; look at them before concluding the model is wrong.

SCALE THE MODEL

Keep every coefficient within a few orders of magnitude. Rescale the units so that money is in thousands rather than pence and time is in hours rather than seconds — mixing units across six orders of magnitude is the usual way a model becomes ill-conditioned.

Avoid very large big-M values (Module 09 §3). **They are simultaneously a numerical problem and a relaxation-strength problem**, which is why tightening them pays twice.

Remove redundant constraints. Presolve does most of this automatically, and helping it — by not writing the same restriction twice in different forms — costs nothing.

And tighten variable bounds wherever the problem justifies it, which improves the conditioning and the relaxation together. **This is CSCE 620 Module 01's robustness lesson and CSCE 633 Module 02's feature scaling arriving for a third time — the same underlying fact about floating-point arithmetic, surfacing in three different subjects**, which at this point in the program should be read as a general property rather than a coincidence.

3 Exact or heuristic

Situation	Approach	Why
The problem is convex	An exact solver.	Polynomial time, a certificate of optimality, and no reason whatsoever to do anything else (Module 01 §1).
Integer, and the solver terminates	Exact, with a gap tolerance of perhaps 1%.	You get a proven bound — take it (Module 09 §1).
Integer, and the solver does not terminate	Reformulate before anything else.	Formulation strength beats every other intervention (Module 09 §4) — tighten big-Ms, break symmetry, consider a different variable choice (Module 12 §1).
Still intractable after reformulation	A heuristic, plus the LP bound computed separately.	Report the gap you achieved — see below.
Black-box objective, expensive to evaluate	Bayesian optimisation or a derivative-free method.	No gradients available and each evaluation costs real money or time (CSCE 633 Module 04 §3).
An answer is needed in milliseconds	A heuristic, or precompute a policy offline.	Real-time dispatch, routing, and control — where an optimal answer that arrives late is worth nothing (CSCE 678 Module 06 §4's argument, again).

Even when you use a heuristic, compute the LP bound. It costs a single additional solve of a relaxation you have already written, and it converts 'this is our answer' into 'this is within 7% of the best possible answer' — a categorically stronger claim. This advice is cheap and almost nobody follows it, which means a great deal of heuristic optimisation is reported with no idea of how good it is.

4 Claiming honestly

- 'Optimal for this model, proved, with a reported gap of 0.0%.' Note *for this model* — which is the load-bearing phrase, and which Module 12 §4 is entirely about.
- 'Within 2.3% of optimal; the bound comes from the LP relaxation after 600 seconds.' The gap, its source, and the budget that produced it.
- 'The binding constraints are capacity at sites 3 and 7; relaxing site 3 by one unit is worth £340 per week.' The dual variables, interpreted in the problem's own vocabulary (Module 04 §2) — which is frequently more valuable to the problem owner than the solution itself.
- 'This model assumes demand is known exactly. Under a 20% demand error, the recommended solution costs 15% more than the robust alternative.' The assumption and its consequence (Module 12 §4).
- And what you cannot honestly say: 'optimal', unqualified, as though the model were the problem. 'Optimal for this model' is the whole discipline of Module 12 compressed into four words, and the difference between the two phrasings is where optimisation projects fail.

WHERE THIS LEAVES YOU

You can recognise convexity and say what it guarantees, construct and interpret a dual, formulate an integer program that actually solves, derive an approximation guarantee from a relaxation, and recognise when a combinatorial problem has exploitable structure.

And you know what the previous two courses were doing. Why gradient descent converges and at what rate; why the condition number governs it; why regularisation is a prior and simultaneously a conditioning improvement; why the lasso produces exact zeros; and why kernels follow from taking the dual rather than being an independent invention.

CSCE 633 supplied the evaluation discipline, CSCE 636 supplied the representations, and this course supplied the theory underneath both — which is why the semester was ordered as it was, with the practice first and the explanation after.

And the closing rule has not changed across nine consecutive courses: state what you measured, state what you assumed, and never claim more than you established. In this course it takes the form 'optimal for this model' — four words that carry the entire difference between a result and an overreach.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Mittelmann — Benchmarks for Optimization Software (free)	Independent, current solver benchmarks across every problem class — the honest basis for the §1 table.
CVXPY, JuMP, and Pyomo documentation (free)	The §1 modelling layers, with examples. JuMP's documentation is unusually good on solver interchange.
Google OR-Tools — CP-SAT documentation (free)	The §1 constraint programming option , which is under-known and frequently wins on scheduling.
Klotz & Newman — Practical Guidelines for Solving Difficult Linear Programs (free)	The §2 numerical diagnostics , from people who support solver users for a living.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Solve the same model with three different solvers** and compare times and reported objectives.
- 2 Deliberately construct an ill-conditioned model** with coefficients spanning eight orders of magnitude, and observe the solvers disagree.

- 3 **Rescale it** and show the disagreement resolves.
- 4 Check the solver's reported condition estimate before and after.
- 5 **Solve a scheduling problem with a MIP solver and with CP-SAT** and compare.
- 6 **Implement a local search heuristic** for a problem you cannot solve exactly.
- 7 **Compute the LP bound separately** and report your heuristic's gap.
- 8 Take a result you produced earlier this semester and **rewrite the claim** to meet §4's standard.
- 9 Find an optimisation claim in a paper or a vendor document and assess whether 'optimal' was qualified.
- 10 **Project 2 is now due.** Submit the formulation, the relaxation and gap, the branch-and-bound results with and without your cuts, the approximation or heuristic comparison, and the honest claim.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the layers of the optimisation stack and an option for each.
- 2 Give three reasons to use a modelling layer.
- 3 Name five symptoms of numerical trouble.
- 4 What is the usual cause, and what is the fix?
- 5 Give six situations and the exact-or-heuristic decision for each.
- 6 Why compute the LP bound even when using a heuristic?
- 7 Give four honest claims and the one thing you cannot say.